

Module 1: Container Security Fundamentals

— Complete Metadata

Module Overview

Field	Value
Module Number	1
Module Title	Container Security Fundamentals
Total Duration	~34:30 (including module summary)
Number of Videos	5 lessons + 1 module summary
Difficulty Level	Beginner-Intermediate
Assessment	3 module MCQs + 2 final assessment MCQs

Module Summary Video

Field	Value
Title	Module 1 Overview: Container Security Fundamentals
File	module_summary_script.md
Duration	2:30
Description	Introduces the module objectives, explains why container security fundamentals matter, lists prerequisites, and previews what students will be able to do after completing all five lessons.
Learning Objectives	Understand the module scope; Identify prerequisites; Preview key skills to be acquired
Prerequisites	None (module entry point)
Resources	Course syllabus, Lab environment setup guide
Screen Mix	PPT 75%, Terminal 15%, Browser 10%
Resolution	1920x1080 @ 30fps

Video 1.1: Container Architecture and Isolation

Field	Value
Title	Container Architecture and Isolation from a Security Perspective
File	video_01_script.md
Duration	8:00
Description	Examines container technology at the kernel level — namespaces for visibility isolation, cgroups for resource control, and layered filesystems for storage. Includes hands-on demonstrations of building container isolation using <code>unshare</code> , inspecting namespaces via <code>/proc</code> , and examining cgroup enforcement. Demonstrates the security risk of data persistence in image layers.
Learning Objectives	1. Explain how namespaces, cgroups, and overlay filesystems provide container isolation; 2. Create isolated process environments using <code>unshare</code> ; 3. Inspect namespace boundaries using <code>/proc/[pid]/ns/</code> and <code>lsns</code> ; 4. Identify the security risk of data persistence in container image layers
Prerequisites	Basic Linux command line proficiency
Resources	<code>man unshare</code> , <code>man namespaces</code> , <code>man cgroups</code> , Docker documentation on storage drivers
Key Commands	<code>unshare</code> , <code>lsns</code> , <code>ls /proc/[pid]/ns/</code> , <code>docker inspect</code> , <code>cat /sys/fs/cgroup/...</code> , <code>docker save</code> , <code>docker history</code>
Screen Mix	PPT 30%, Terminal 45%, Code Editor 15%, Browser 10%
Resolution	1920x1080 @ 30fps

Video 1.2: Container vs. VM Security Models

Field	Value
Title	Container vs. VM Security Models
File	video_02_script.md
Duration	7:00
Description	Compares container and virtual machine architectures from a security perspective. Demonstrates that all containers share the host kernel regardless of base image. Analyzes real CVEs (VENOM for VM escape, CVE-2019-5736 for container escape) to illustrate complexity differences. Provides a security decision matrix for choosing appropriate isolation technology.
Learning Objectives	1. Compare attack surfaces of containers vs. VMs across six dimensions; 2. Demonstrate the shared kernel using multi-distribution container testing; 3. Analyze historical escape CVEs and their complexity; 4. Apply a decision matrix for isolation technology selection
Prerequisites	Video 1.1
Resources	NVD entries for CVE-2019-5736, CVE-2015-3456; CNCF Container Security whitepaper
Key Commands	<code>uname -r</code> , <code>docker run</code> , <code>dmesg</code> , <code>lsmod</code> , <code>cat /proc/version</code> , <code>cat /proc/cpuinfo</code>
Screen Mix	PPT 35%, Terminal 35%, Browser 20%, Code Editor 10%
Resolution	1920x1080 @ 30fps

Video 1.3: The Shared Kernel Problem

Field	Value
Title	The Shared Kernel Problem
File	video_03_script.md
Duration	7:00
Description	Deep-dives into the shared kernel as the primary container security risk. Quantifies the system call attack surface, analyzes CVE-2022-0185 as a case study, demonstrates cross-container visibility from the host perspective, and explains how kernel modules bypass all container isolation. Introduces mitigation strategies covered in later modules.
Learning Objectives	1. Quantify the system call attack surface available to containers; 2. Analyze a real kernel CVE and its container impact; 3. Demonstrate cross-container information exposure via the host; 4. Explain how kernel modules bypass container isolation; 5. Enumerate available mitigations and their limitations
Prerequisites	Videos 1.1, 1.2
Resources	NVD entry for CVE-2022-0185; Linux syscall table reference; Docker default Seccomp profile
Key Commands	<code>ausyscall --dump</code> , <code>docker inspect</code> , <code>cat /proc/[pid]/environ</code> , <code>nsenter</code> , <code>lsmod</code>
Screen Mix	PPT 25%, Terminal 40%, Code Editor 20%, Browser 15%
Resolution	1920x1080 @ 30fps

Video 1.4: Docker Security Basics and Daemon Configuration

Field	Value
Title	Docker Security Basics and Daemon Configuration
File	video_04_script.md
Duration	8:00
Description	Examines Docker-specific security: daemon architecture, socket permissions, and configuration hardening. Demonstrates that Docker socket access equals root access. Compares insecure and hardened daemon.json configurations. Shows Shodan results for exposed Docker daemons. Covers default capability analysis, Docker Content Trust, and automated auditing with Docker Bench for Security.
Learning Objectives	1. Explain why Docker socket access is equivalent to root access; 2. Audit and harden the Docker daemon configuration file; 3. Identify exposed Docker daemons using Shodan; 4. Analyze and minimize default Linux capabilities; 5. Configure Docker Content Trust for supply-chain security; 6. Run Docker Bench for Security and interpret results
Prerequisites	Videos 1.1–1.3
Resources	CIS Docker Benchmark; Docker daemon configuration reference; Docker Content Trust documentation
Key Commands	<code>ls -la /var/run/docker.sock</code> , <code>docker run -v /:/hostfs</code> , <code>capsh --decode</code> , <code>docker trust</code> , Docker Bench for Security
Screen Mix	PPT 20%, Terminal 35%, Code Editor 30%, Browser 15%
Resolution	1920x1080 @ 30fps

Video 1.5: Container Attack Surface Mapping

Field	Value
Title	Container Attack Surface Mapping
File	video_05_script.md
Duration	7:00
Description	Maps the complete container attack surface across five layers: application, image, runtime, orchestration, and host/kernel. Demonstrates application-level attacks using DVWA, image vulnerability scanning with Trivy, identifies dangerous docker run flags, discusses supply chain attack vectors, and previews Kubernetes attack surface. Provides a comprehensive security checklist for the module.
Learning Objectives	1. Map the five-layer container attack surface model; 2. Scan container images for vulnerabilities using Trivy; 3. Identify dangerous Docker runtime flags and their secure alternatives; 4. Describe container supply chain attack vectors; 5. Recognize orchestration-layer security concerns
Prerequisites	Videos 1.1–1.4
Resources	Trivy documentation; OWASP Container Security guide; Docker security best practices; MITRE ATT&CK for Containers
Key Commands	trivy image , docker run (various flags), Trivy filesystem scanning
Screen Mix	PPT 25%, Terminal 35%, Browser 25%, Code Editor 15%
Resolution	1920x1080 @ 30fps

Technical Production Requirements

Requirement	Specification
Resolution	1920x1080 Full HD
Frame Rate	30fps constant
Terminal Theme	Dark (Dracula or One Dark), 150% zoom
Code Editor	VS Code, 150% zoom, max 20-22 lines visible
PPT Slides	Maximized, 150% for readability
Browser	Chromium/Chrome, 150% zoom, pre-loaded pages
Screen Mix (Module Avg)	Terminal 38%, PPT 27%, Code Editor 18%, Browser 17%
Host OS	Ubuntu 22.04 LTS
Docker Version	24.x+ with Docker Compose v2
Pre-pulled Images	alpine, ubuntu:22.04, centos:7, nginx, nginx:alpine, vulnerables/web-dvwa, aquasec/trivy, docker/docker-bench-security